康轨 KangRail

业务流程图（9 张）

从零看懂业务层

图 1 • 系统分层架构与依赖方向
代码写在哪、模块之间谁能依赖谁

图 2 • 一次 HTTP 请求的完整生命周期
拦截器 → Controller → Service → 数据源 → 返回

图 3 • 登录鉴权全流程
上半：登录怎么签发 token；下半：后续请求怎么校验 token

图 4 • 余票查询与「缓存三兄弟」
缓存穿透 / 击穿 / 雪崩 的完整对策

图 5 • 购票主流程（核心链路）
Redisson 锁 + Lua 预扣 + MQ 异步落库 —— 整个项目最重要的一张图

图 6 • 订单状态机
状态流转 + 为什么状态判断必须放进 UPDATE 的 WHERE 条件

图 7 • MQ 异步链路与超时关单
上半：异步落库（削峰）；下半：TTL + 死信队列实现延时消息

图 8 • 取消订单与退票
库存回补的两条路径 + 一个必须注意的事务陷阱

图 9 • 数据一致性全景
MySQL / Redis / RabbitMQ 三者关系与一致性等级

说明：

· 这些图是真实 PNG 文件，放在 docs/diagrams/ 目录下。

· 网页版 Markdown（语雀 / 飞书 / Notion）不解析相对路径，
所以那边看不到图 —— 用浏览器打开 docs/diagrams/index.html，
或者直接看本 PDF。

· 想改图：编辑 docs/diagrams/make_diagrams.py 后重跑该脚本。

图 1 · 系统分层架构与依赖方向

代码写在哪、模块之间谁能依赖谁

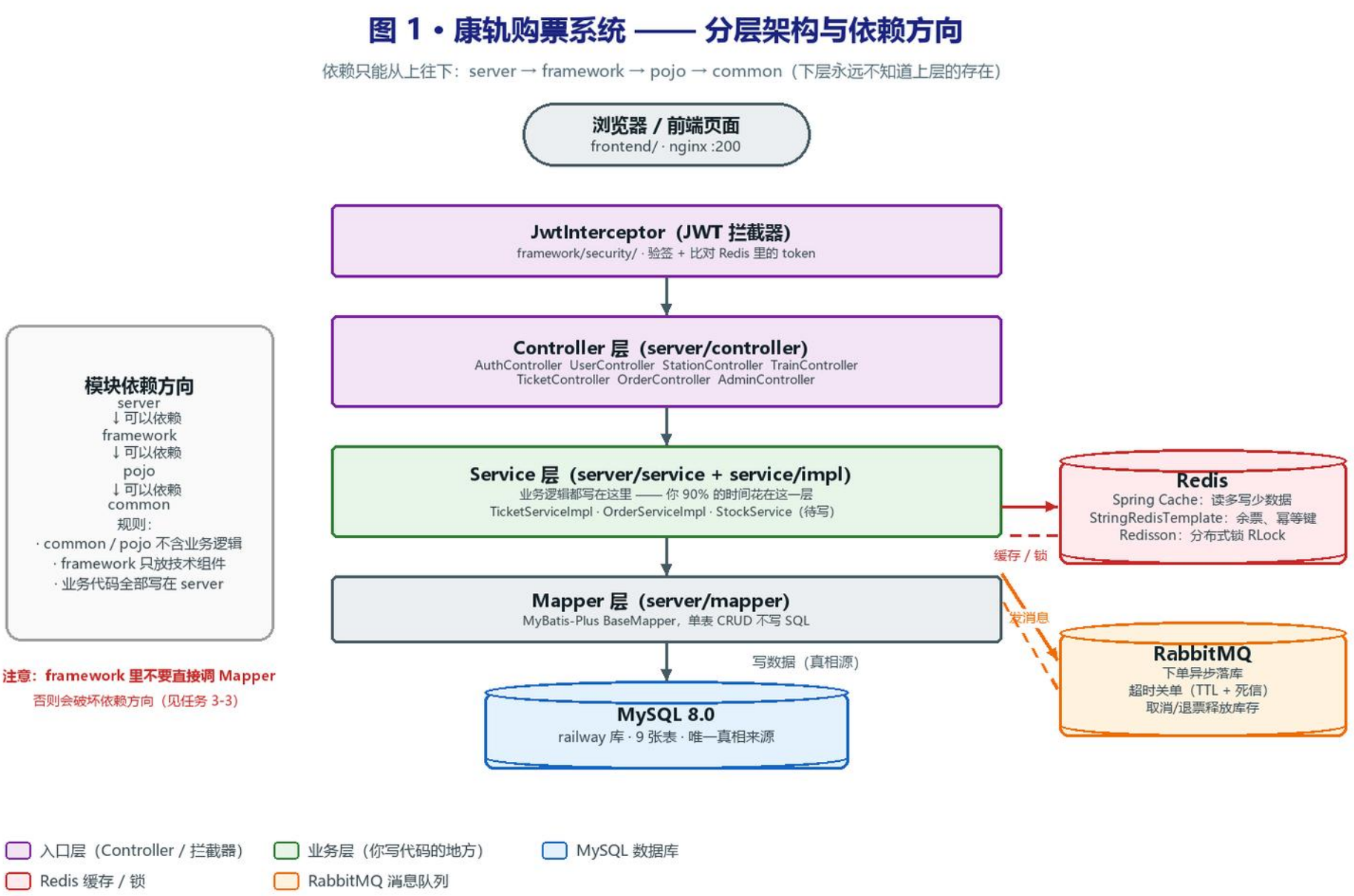


图 2 • 一次 HTTP 请求的完整生命周期

拦截器 → Controller → Service → 数据源 → 返回

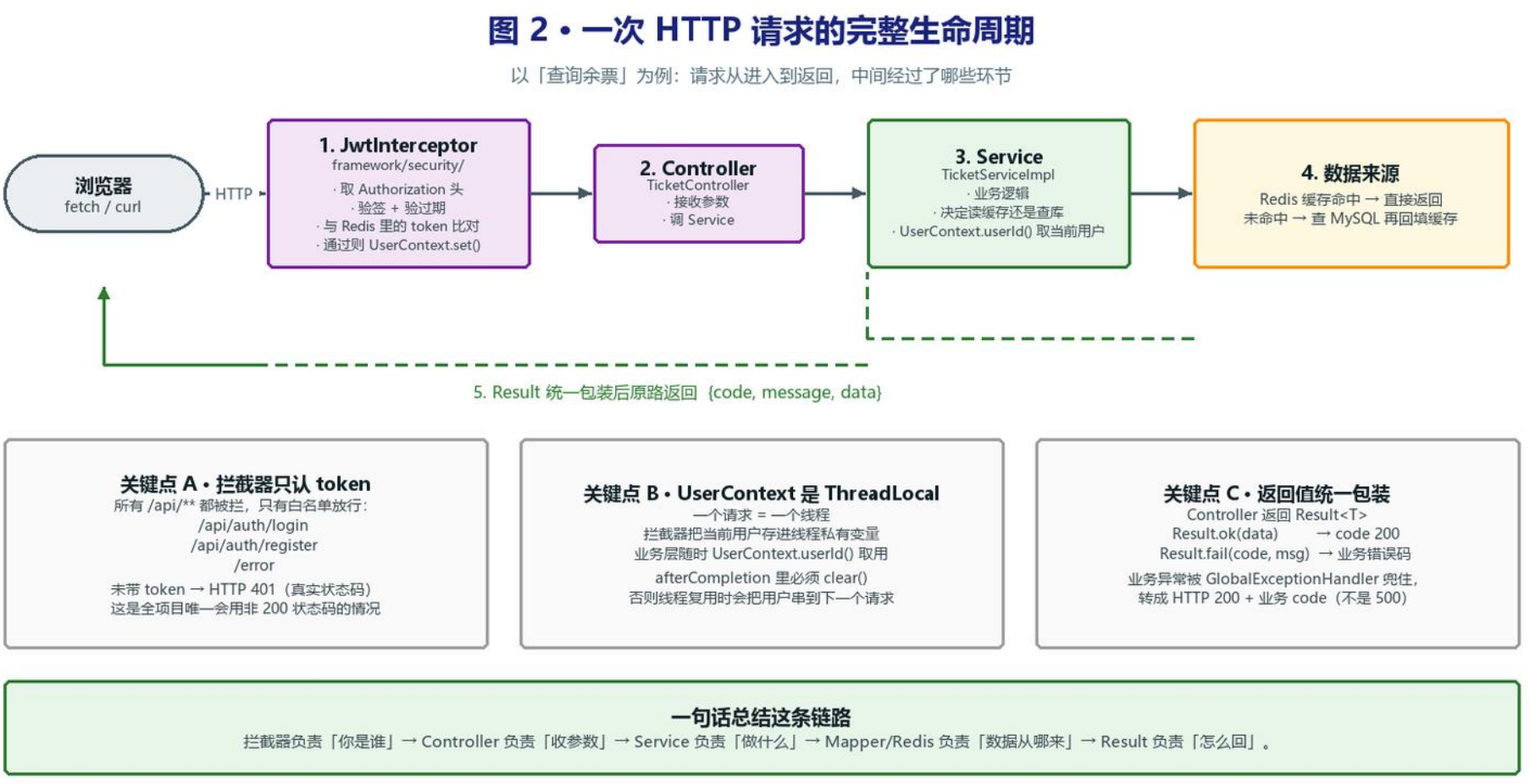


图 3 • 登录鉴权全流程

上半：登录怎么签发 token；下半：后续请求怎么校验 token

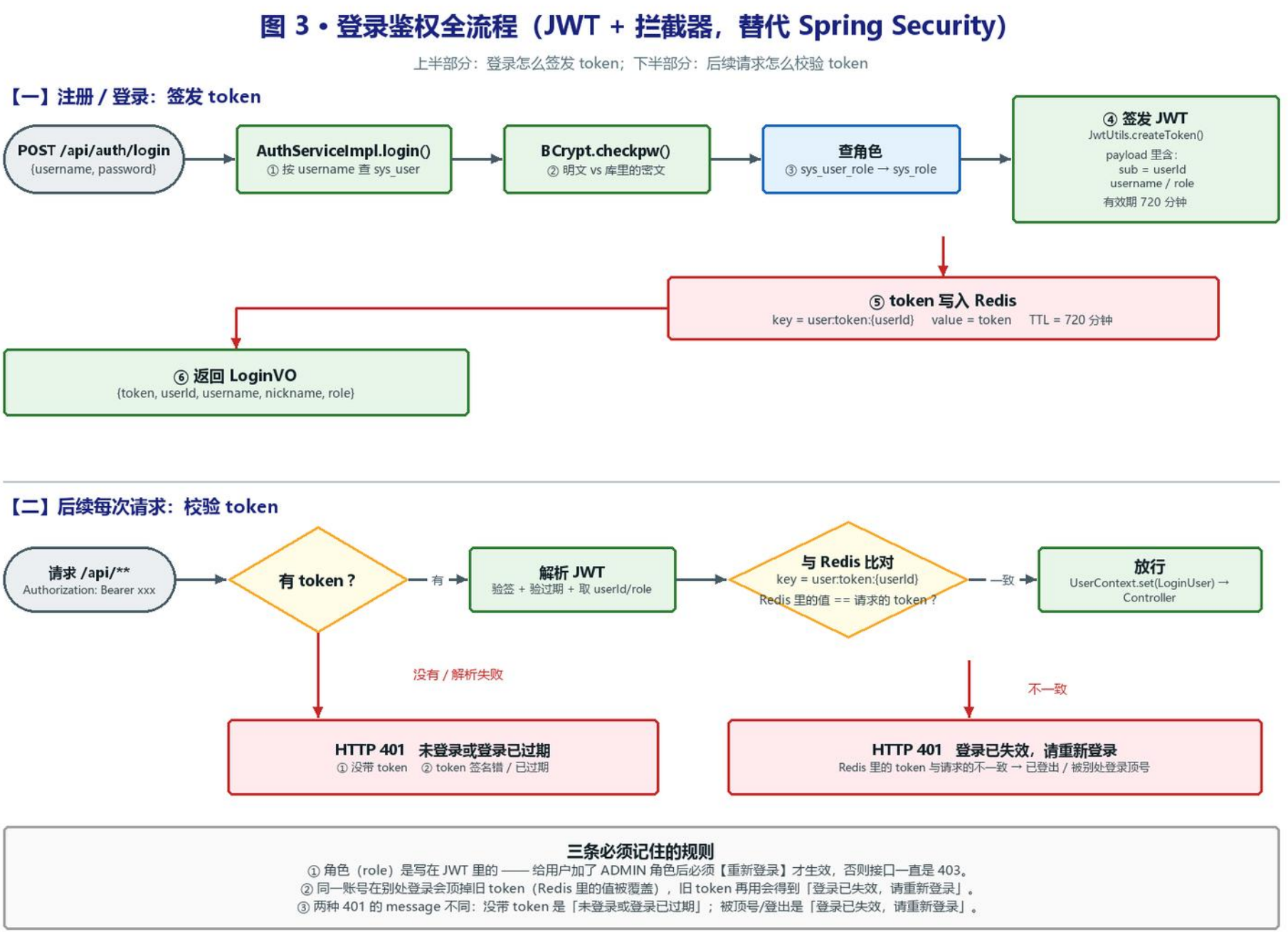


图 4 • 余票查询与「缓存三兄弟」

缓存穿透 / 击穿 / 雪崩 的完整对策

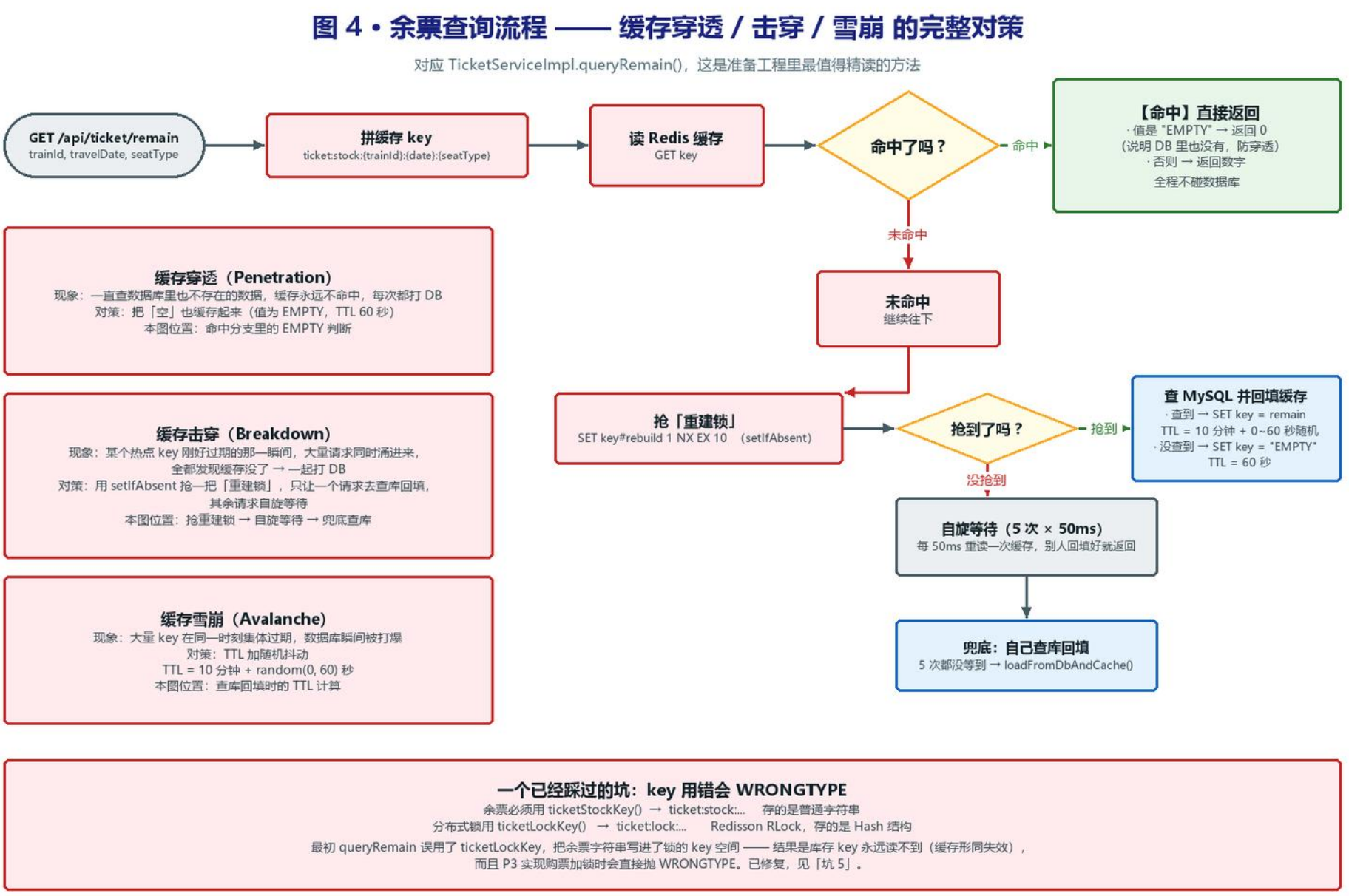
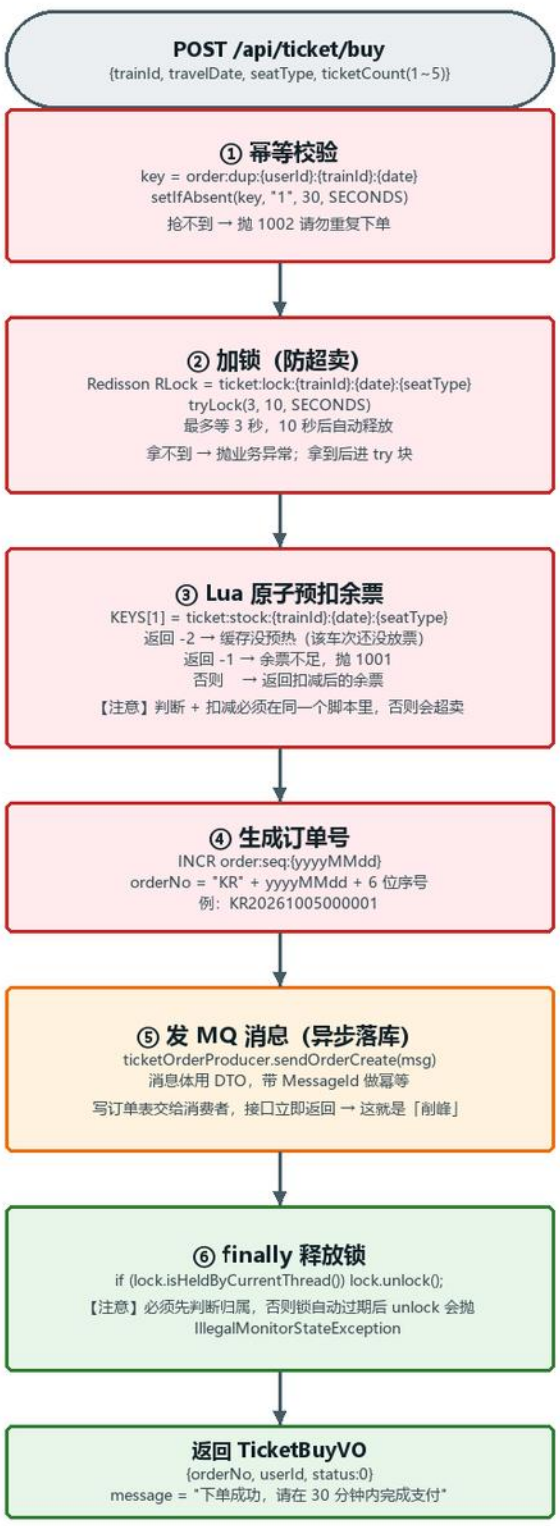


图 5 • 购票主流程（核心链路）

Redisson 锁 + Lua 预扣 + MQ 异步落库 —— 整个项目最重要的一张图

图 5 • 购票主流程（核心链路）—— Redisson 锁 + Lua 预扣 + MQ 异步落库

对应 TicketServiceImpl.buy()，这是整个项目最重要的方法（任务 3-1）



这条链路要解决的核心问题：不超卖 + 抗峰值

库存 5 张票，20 个人同时抢 —— 必须正好成功 5 个，余票不能为负，订单表不能有重复订单号

为什么 ① 幂等不能少

用户手抖点两次「提交订单」，不加幂等键就会生成两个订单。

为什么 ② 分布式锁不能少

synchronized 只在单个 JVM 内有效。应用部署 2 个实例时，
两个 JVM 各有一把自己的锁，仍然会同时扣库存。

为什么 ③ 必须用 Lua

如果写成「先 GET 判断，再 DECRBY 扣减」两条命令，
两个请求可能同时判断「够」，然后都扣 → 超卖。
Lua 脚本在 Redis 里原子执行，中途不会被插队。

为什么 ⑤ 要异步落库

同步写订单表会让接口响应变慢，扛不住峰值。
先扣库存并立即返回，写库交给消费者慢慢做。

为什么 ⑥ 要判断 isHeldByCurrentThread

如果业务执行超过了锁的 10 秒 TTL，锁已经自动释放，
此时直接 unlock() 会抛 IllegalMonitorStateException。

× 错误写法（会超卖）

```
String stock = redis.get(key);
if (Integer.parseInt(stock) >= count) {
    redis.decrBy(key, count);
}
```

两个线程同时判断「够」，然后都扣 → 超卖

√ 正确写法（Lua 原子）

```
local stock = redis.call('GET', KEYS[1])
if not stock then return -2 end
if tonumber(stock) < tonumber(ARGV[1]) then return -1 end
return redis.call('DECRBY', KEYS[1], ARGV[1])
```

判断和扣减在一个脚本里，Redis 保证原子性 —— 不可能超卖

图 6 • 订单状态机

状态流转 + 为什么状态判断必须放进 UPDATE 的 WHERE 条件

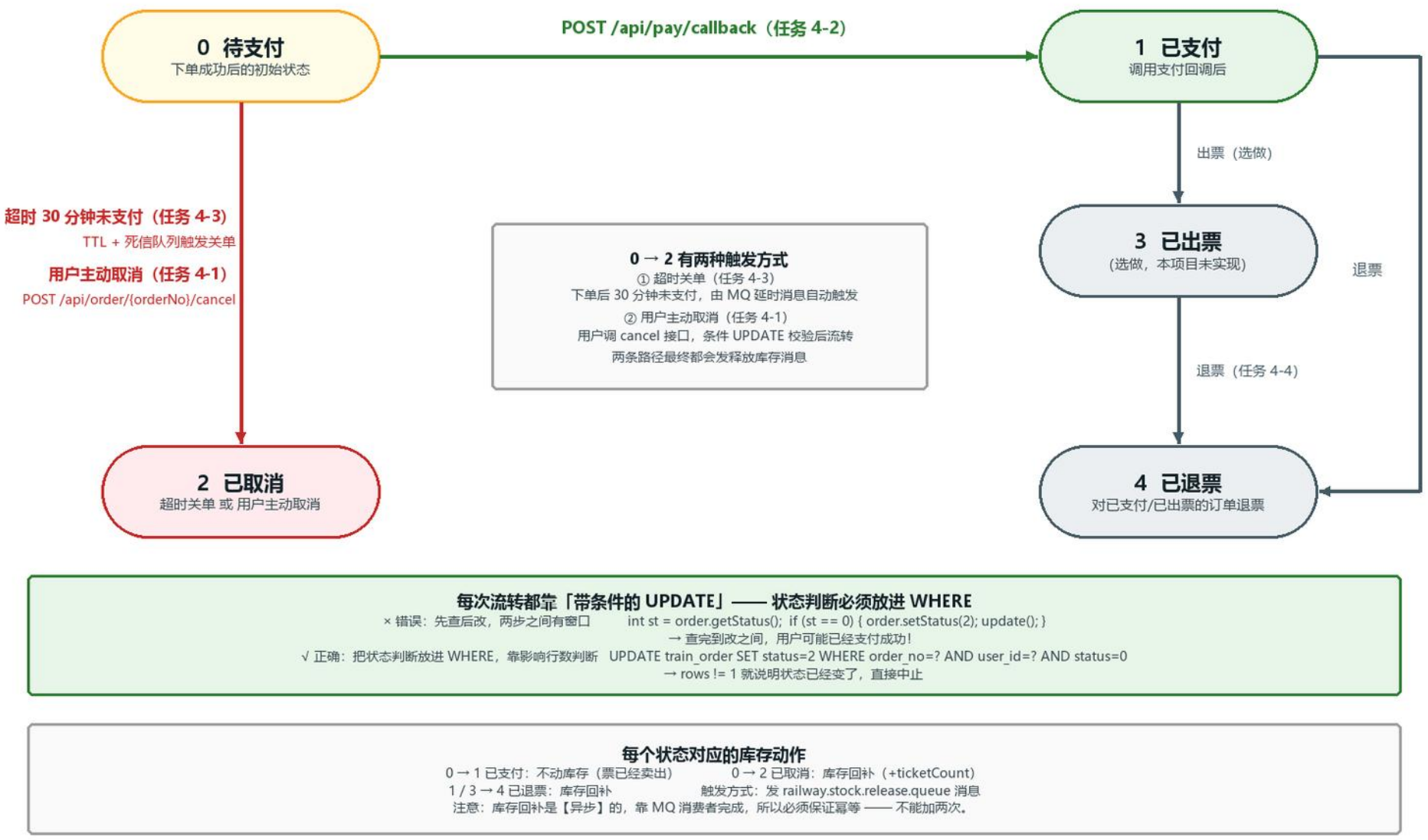


图 7 • MQ 异步链路与超时关单

上半：异步落库（削峰）；下半：TTL + 死信队列实现延时消息

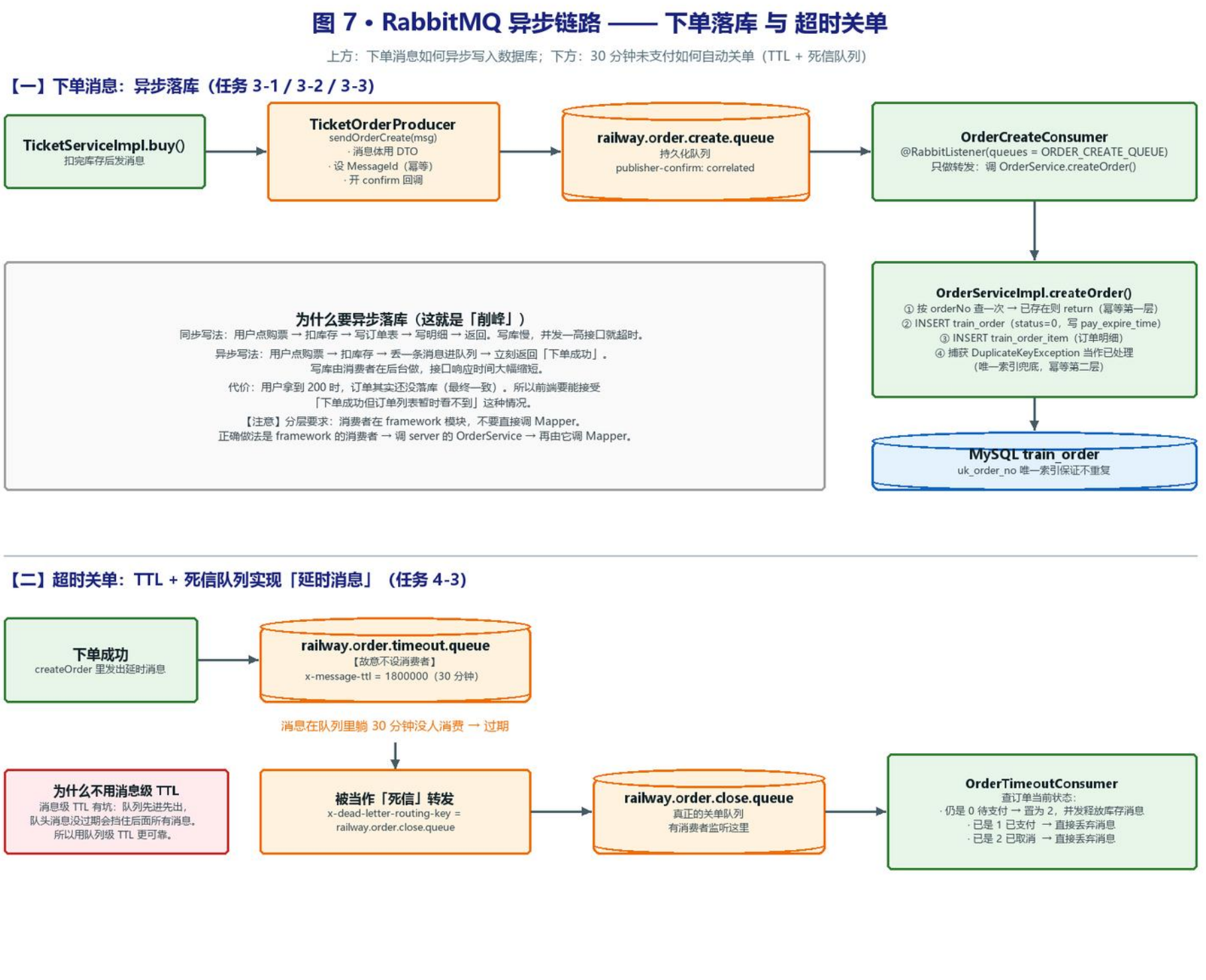


图 8 • 取消订单与退票

库存回补的两条路径 + 一个必须注意的事务陷阱



图 9 • 数据一致性全景

MySQL / Redis / RabbitMQ 三者关系与一致性等级

